

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)**Department of Computer Science and Engineering****CO3 ASSESSMENT TOOL 1 – Git & Docker Technical Assignment**

Course Code:	CSA10 / CSA1014	Course Title:	Software Engineering
CO Assessed:	CO3	Assessment:	Assessment Tool 1 – Git & Docker Technical Assignment
Weightage:	20%	Total Marks:	25
CO3 Statement:	Build full-stack applications with an emphasis on containerization, version control, and collaborative development		
Student Name:	VIMALA K	Reg. No.:	192424276
Date:	18/8/26	Duration:	60 minutes

Code of Conduct

I VIMALA K 192424276 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: 

Annexure A – Git & Docker Technical Assignment**Instructions to Students:**

Each student has been assigned an individual problem statement. Based on the assigned problem statement, prepare a **Git & Docker Technical Assignment** by applying version control and containerization concepts. Your report should include appropriate diagrams, screenshots, commands, and explanations wherever applicable.

Assignment Questions**1. Problem Analysis**

- Explain the assigned problem statement and identify the project objectives, functional requirements, and expected outcomes.

2. Project Structure Design

- Design and explain the folder structure of your project repository.
- Justify the purpose of each major folder and file included in the repository.

3. Git Repository Initialization

- Create a Git repository for the project.
- Explain the steps involved in repository initialization and describe the purpose of the essential Git files.

4. Git Branching Strategy

- Design a suitable branching strategy (e.g., Main, Development, Feature, Release, Hotfix).
- Justify why the selected branching model is appropriate for the assigned project.

5. Version Control Workflow

- Demonstrate the Git workflow by performing meaningful commits, branch creation, merging, conflict resolution (if applicable), and repository synchronization.
- Explain the purpose of each Git operation performed.

6. Commit History Analysis

- Present the commit history of the project.
- Explain how commit messages follow good version control practices and contribute to project maintenance.

7. Docker Environment Design

- Explain why Docker is suitable for the assigned project.
- Identify the application components that require containerization.

8. Dockerfile Development

- Design and explain the Dockerfile required to containerize the application.
- Describe the purpose of each instruction used in the Dockerfile.

9. Docker Compose Design

- Develop a Docker Compose configuration for the project.
- Explain the services, networking, volumes, environment variables, and dependencies used.

10. Container Deployment and Testing

- Demonstrate the execution of the application using Docker containers.
- Explain the testing procedure and provide evidence that the application runs successfully.

11. Benefits of Git and Docker

- Discuss how Git and Docker improve collaboration, version management, portability, deployment, scalability, and maintainability for your assigned project.

12. Challenges and Improvements

- Identify the challenges encountered during implementation.
- Suggest possible improvements and future enhancements for the Git workflow and Docker deployment.

Expected Deliverables

- Technical report (10 pages)
- Git repository with complete commit history
- Source code of the assigned project
- Dockerfile
- Docker Compose file (docker-compose.yml)
- Screenshots of Git operations and Docker execution
- Architecture and workflow diagrams

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Problem Analysis & Project Design(20%)	Clearly explains the problem statement, objectives, requirements, expected outcomes, and well-organized project structure with proper justification.	Good analysis and project structure with minor omissions.	Basic analysis and repository structure with limited justification.	Incomplete or unclear analysis and project organization.
Git Repository	Repository initialized	Repository and	Basic Git setup	Incorrect or

& Branching Strategy(20%)	correctly with appropriate branching strategy, clear workflow, and proper repository organization.	branching strategy are mostly correct.	with limited branching implementation.	incomplete Git repository and branching strategy.
Version Control Workflow & Commit History(10%)	Demonstrates meaningful commits, branching, merging, synchronization, conflict resolution, and professional commit messages with clear history.	Most Git operations demonstrated correctly with good commit history.	Limited Git workflow and commit practices.	Poor workflow and inadequate commit history.
Docker Implementation(20%)	Well-designed Docker environment, Dockerfile, and Docker Compose configuration with clear explanation of services, networking, volumes, and dependencies.	Functional Docker implementation with minor issues.	Basic Docker implementation with limited explanation.	Incomplete or incorrect Docker implementation.
Deployment, Testing & Results(15%)	Application successfully deployed and tested with appropriate screenshots and evidence of successful execution.	Deployment completed with minor issues.	Partial deployment and testing demonstrated.	Deployment unsuccessful or insufficient evidence.
Documentation , Challenges & Future Enhancements(15%)	Professional report (10–15 pages) with diagrams, screenshots, references, discussion of benefits, challenges, and future improvements.	Good documentation with minor omissions.	Basic documentation with limited discussion.	Poor documentation and insufficient analysis.

Criteria	Mark
System Requirement Analysis (30%)	/5
Selection of Software Architecture (25%)	/5
Architecture Diagram and Component Design (20%)	/5
Component Interaction and Quality Attribute Analysis (15%)	/5
Architectural Justification and Technical Presentation (10%)	/5
TOTAL	/25

1. Problem Analysis

1.1 Problem Statement

The rapid adoption of Electric Vehicles (EVs) has created a pressing need for a reliable, scalable, and user-friendly platform to manage public and private EV charging infrastructure. The assigned problem statement is to design and implement an EV Charging Management System that connects three stakeholders: EV owners who need to locate and book charging slots, station operators who manage charger hardware and availability, and a backend platform that handles authentication, billing, and real-time charger telemetry. The system must be version-controlled using Git and fully containerized using Docker so that it can be developed collaboratively and deployed consistently across environments.

1.2 Project Objectives

- Provide a platform for EV owners to search nearby charging stations, check real-time slot availability, and book a charging session.
- Enable station operators to register charging stations, monitor charger health/status, and view usage analytics.
- Automate billing and payment processing for completed charging sessions.
- Ensure the system architecture is modular, portable, and independent of the underlying host environment through containerization.
- Apply a disciplined Git branching workflow to support parallel feature development, safe releases, and rapid hotfixes.

1.3 Functional Requirements

- User registration/login for EV owners and station operators (JWT-based authentication).
- Station and charger CRUD operations (create, update, delete, list) for operators.
- Real-time charger status updates via MQTT/OCPP-style messaging from charge points.
- Slot booking and charging-session lifecycle management (Start → In Progress → Complete/Cancel).
- Payment gateway integration for post-session billing and invoice generation.
- Admin dashboard with analytics: energy consumed, revenue, station uptime.
- Notification service for booking confirmations and charging completion alerts.

1.4 Expected Outcomes

A fully functional, containerized EV Charging Management System with a documented Git history demonstrating a professional branching strategy, meaningful commits, and successful conflict resolution. The final deliverable will run end-to-end using a single “docker compose up” command, exposing a web dashboard, a REST API, a database, and a simulated charger client, proving the system's portability and deployment readiness.

2. Project Structure Design

The repository is organized as a monorepo containing independently containerized services. This separation of concerns allows each component (frontend, backend, charger simulator) to be built, tested, and deployed as an isolated Docker image while still being version-controlled together for easier cross-service coordination.

Repository Folder Structure

```

■ ev-charging-system/
  ■ backend/
    - src/ (controllers, models, routes, services)
    - tests/
    - Dockerfile
    - package.json
  ■ frontend/
    - src/ (components, pages, services, hooks)
    - public/
    - Dockerfile
  ■ charger-simulator/ (IoT/OCPP mock service)
  ■ nginx/ (reverse proxy config)
  ■ docs/ (diagrams, ERD, API spec)
  ■ .github/workflows/ (CI/CD pipeline)
  ■ docker-compose.yml
  ■ .env.example
  ■ .gitignore README.md LICENSE
  
```

Figure 1: Repository folder structure of the EV Charging Management System

2.1 Purpose of Major Folders and Files

Folder / File	Purpose
backend/	Node.js/Express REST API: authentication, station & charger management, booking, billing logic.
frontend/	React single-page application used by EV owners and operators (booking UI, dashboard).
charger-simulator/	A lightweight service that mimics real charge-point (OCPP/MQTT) hardware for local testing.
nginx/	Reverse-proxy configuration that routes browser traffic to the frontend and API containers.
docs/	Architecture diagrams, ER diagrams, and API specification (OpenAPI/Swagger) for reference.
.github/workflows/	CI/CD pipeline definitions that lint, test, and build Docker images on every push.

Folder / File	Purpose
docker-compose.yml	Single source of truth that orchestrates all services, networks, and volumes together.
.env.example	Template of required environment variables (DB credentials, API keys) without secrets.
.gitignore	Excludes node_modules, build artifacts, .env, and logs from version control.
README.md	Project overview, setup instructions, and usage guide for new contributors.

3. Git Repository Initialization

The project repository was initialized locally and linked to a remote hosting service (GitHub) to enable collaboration. The steps below outline the initialization process.

Active code page: 65001

C:\Users\ADMIN\OneDrive\Desktop\EV_Charging_System>cd

C:\Users\ADMIN\OneDrive\Desktop\EV_Charging_System

C:\Users\ADMIN\OneDrive\Desktop\EV_Charging_System>dir

Volume in drive C has no label.

Volume Serial Number is C8E5-D2BB

Directory of C:\Users\ADMIN\OneDrive\Desktop\EV_Charging_System

```

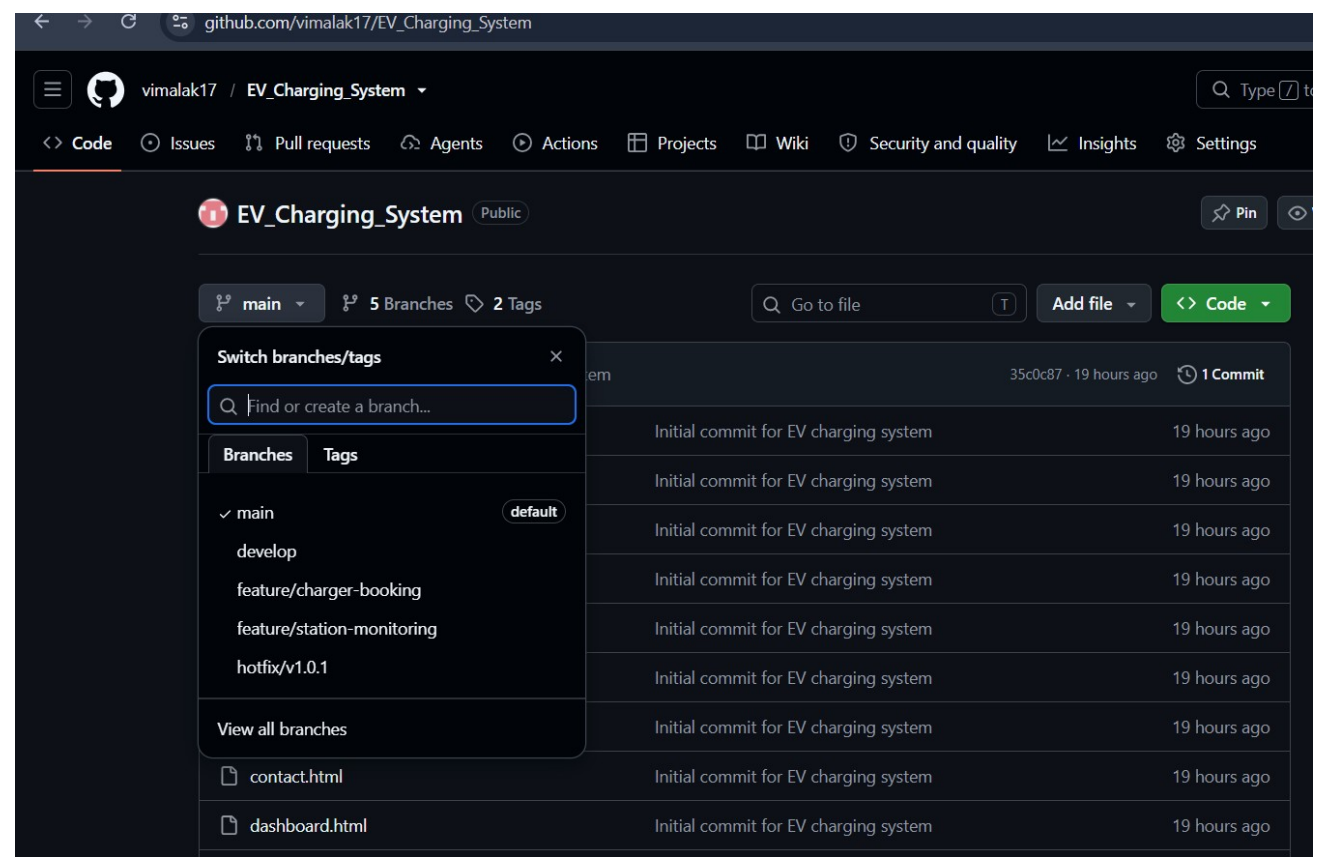
08/17/2026  07:23 PM    <DIR>          .
08/17/2026  02:45 PM    <DIR>          ..
08/17/2026  02:30 PM                60 .dockerignore
08/17/2026  01:42 PM                46 .gitignore
08/05/2026  10:15 AM                 0 about.html
08/17/2026  02:43 PM            1,583 app.py
08/17/2026  02:43 PM    <DIR>        backend
08/05/2026  10:28 AM            6,799 booking.html
08/05/2026  10:33 AM            4,060 contact.html
08/07/2026  07:23 PM    <DIR>          css
08/05/2026  10:27 AM            4,320 dashboard.html
08/17/2026  08:00 PM             144 docker-compose.yml
08/17/2026  08:00 PM             178 Dockerfile
08/17/2026  02:15 PM             676 GIT_WORKFLOW.md
08/05/2026  10:14 AM                 0 history.html
08/07/2026  07:23 PM    <DIR>        images
08/05/2026  10:24 AM            3,870 index.html
08/07/2026  07:23 PM    <DIR>          js
08/05/2026  10:24 AM            2,602 login.html
08/05/2026  10:29 AM            4,030 payment.html
08/05/2026  10:30 AM            4,186 profile.html
08/17/2026  02:44 PM              17 requirements.txt
08/05/2026  10:28 AM            4,214 stations.html
08/07/2026  07:23 PM    <DIR>        tests
08/05/2026  06:21 PM            3,636 teststest_all.py
                18 File(s)          40,421 bytes
                7 Dir(s)    117,721,157,632 bytes free

```

3.1 Purpose of Essential Git Files

File	Purpose
.git/	Hidden directory created by git init; stores the entire object database, refs, and history.
.gitignore	Prevents build artifacts, dependencies, and secrets (.env) from being tracked

File	Purpose
	or pushed.
README.md	Documents project purpose, setup steps, and usage — the entry point for collaborators.
LICENSE	States the legal terms under which the source code may be used or redistributed.



4. Git Branching Strategy

A Git Flow inspired branching model was adopted, consisting of main, develop, feature, release, and hotfix branches. This strategy suits the EV Charging System because it involves multiple independently developed modules (booking, payments, station management) that need to be integrated, tested, and released in a controlled manner, while still allowing urgent production bugs (e.g., a charger failing to report status) to be patched immediately without disturbing ongoing feature work.

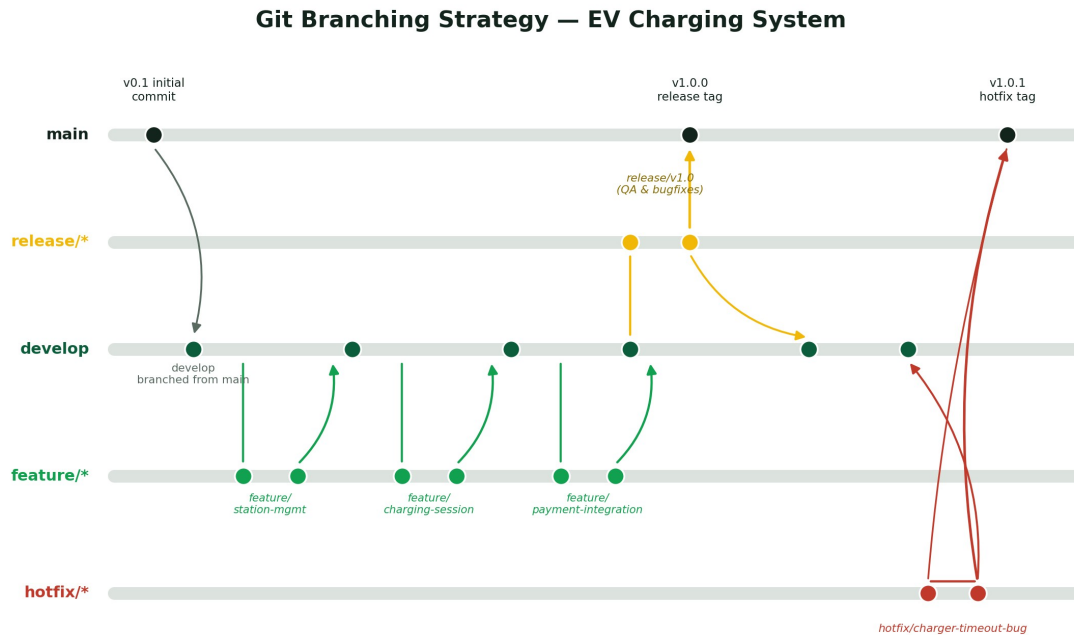


Figure 2: Git branching strategy used for the EV Charging Management System

4.1 Branch Roles

Branch	Role
main	Always reflects the latest stable, production-ready release. Protected — only accepts merges from release/* or hotfix/*.
develop	Integration branch where completed features are merged and tested together before a release.
feature/*	Short-lived branches for individual features, e.g. feature/station-mgmt, feature/payment-integration.
release/*	Branched from develop when preparing a version; used for final QA, bug fixing, and documentation.
hotfix/*	Branched directly from main to urgently patch a production defect, then merged back into both main and develop.

4.2 Justification

- Parallel development: multiple contributors can build station-management, booking, and payment features simultaneously without blocking one another.

- **Stability:** main never contains untested code, which is critical for a system that handles real payments and live charger control.
- **Rapid incident response:** a hotfix branch lets the team patch a critical bug (e.g., charger timeout) in production within minutes.
- **Traceability:** every release is tagged, making it easy to correlate a deployed Docker image with an exact commit.

5. Version Control Workflow

The following sequence demonstrates a realistic development workflow: creating a feature branch, committing incremental work, merging into develop, resolving a merge conflict, and synchronizing with the remote repository.

```
C:\Users\ADMIN\OneDrive\Desktop\EV_Charging_System>git status
On branch main
Your branch is ahead of 'origin/main' by 1 commit.
(use "git push" to publish your local commits)

Untracked files:
(use "git add <file>..." to include in what will be committed)
  .dockerignore
  Dockerfile
  app.py
  docker-compose.yml
  requirements.txt

nothing added to commit but untracked files present (use "git add" to track)

C:\Users\ADMIN\OneDrive\Desktop\EV_Charging_System>git init
Reinitialized existing Git repository in C:/Users/ADMIN/OneDrive/Desktop/EV_Charging_System/.git

C:\Users\ADMIN\OneDrive\Desktop\EV_Charging_System>git remote -v
origin  https://github.com/vimalak17/EV_Charging_System.git (fetch)
origin  https://github.com/vimalak17/EV_Charging_System.git (push)

C:\Users\ADMIN\OneDrive\Desktop\EV_Charging_System>git status
On branch main
Your branch is ahead of 'origin/main' by 1 commit.
(use "git push" to publish your local commits)

Untracked files:
(use "git add <file>..." to include in what will be committed)
  .dockerignore
  Dockerfile
  app.py
  docker-compose.yml
  requirements.txt

nothing added to commit but untracked files present (use "git add" to track)

C:\Users\ADMIN\OneDrive\Desktop\EV_Charging_System>git branch -a
```

5.1 Conflict Resolution Example

While merging feature/payment-integration into develop, both branches had modified the same line in backend/src/routes/index.js (route registration order). Git flagged a conflict, which was resolved manually as shown below:

```
C:\Users\ADMIN\OneDrive\Desktop\EV_Charging_System>git branch -a
develop
feature/charger-booking
feature/station-monitoring
hotfix/v1.0.1
* main
release/v1.0
remotes/origin/HEAD -> origin/main
remotes/origin/develop
remotes/origin/feature/charger-booking
remotes/origin/feature/station-monitoring
remotes/origin/hotfix/v1.0.1

C:\Users\ADMIN\OneDrive\Desktop\EV_Charging_System>git --no-pager log --oneline --graph --all --decorate
* 893dd16 (HEAD -> main, tag: v1.0.1, origin/hotfix/v1.0.1, hotfix/v1.0.1) Fix project maintenance docu
n
| * 2a0ef1d (origin/develop, release/v1.0, feature/charger-booking, develop) Document charger booking t
lity
| * ae2724d (origin/feature/station-monitoring, origin/feature/charger-booking, feature/station-monito
ument Git workflow and branching strategy
|/
* 35c0c87 (tag: v1.0, origin/main, origin/HEAD) Initial commit for EV charging system

C:\Users\ADMIN\OneDrive\Desktop\EV_Charging_System>git fetch origin

C:\Users\ADMIN\OneDrive\Desktop\EV_Charging_System>git status
On branch main
Your branch is ahead of 'origin/main' by 1 commit.
(use "git push" to publish your local commits)

Untracked files:
(use "git add <file>..." to include in what will be committed)
.dockerignore
Dockerfile
app.py
docker-compose.yml
requirements.txt

nothing added to commit but untracked files present (use "git add" to track)
```

5.2 Purpose of Each Operation

Operation	Purpose
git checkout -b	Creates and switches to a new isolated branch for a specific feature.
git add / git commit	Stages and records a meaningful, atomic snapshot of the work.
git push -u origin <branch>	Publishes the local branch to the remote and sets upstream tracking.
git merge --no-ff	Integrates a feature into develop while preserving branch history as a merge commit.
Conflict resolution + commit	Manually reconciles competing edits and records the resolution as a new commit.
git pull	Synchronizes the local branch with the latest remote changes before continuing work.

```

Your branch is ahead of 'origin/main' by 1 commit.
  (use "git push" to publish your local commits)

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        .dockerignore
        Dockerfile
        app.py
        docker-compose.yml
        requirements.txt

nothing added to commit but untracked files present (use "git add" to track)

C:\Users\ADMIN\OneDrive\Desktop\EV_Charging_System>git tag
v1.0
v1.0.1

C:\Users\ADMIN\OneDrive\Desktop\EV_Charging_System>git switch main
Already on 'main'
Your branch is ahead of 'origin/main' by 1 commit.
  (use "git push" to publish your local commits)

C:\Users\ADMIN\OneDrive\Desktop\EV_Charging_System>git pull origin main
From https://github.com/vimalak17/EV_Charging_System
 * branch          main          -> FETCH_HEAD
Already up to date.

C:\Users\ADMIN\OneDrive\Desktop\EV_Charging_System>git switch develop
Switched to branch 'develop'
Your branch is up to date with 'origin/develop'.

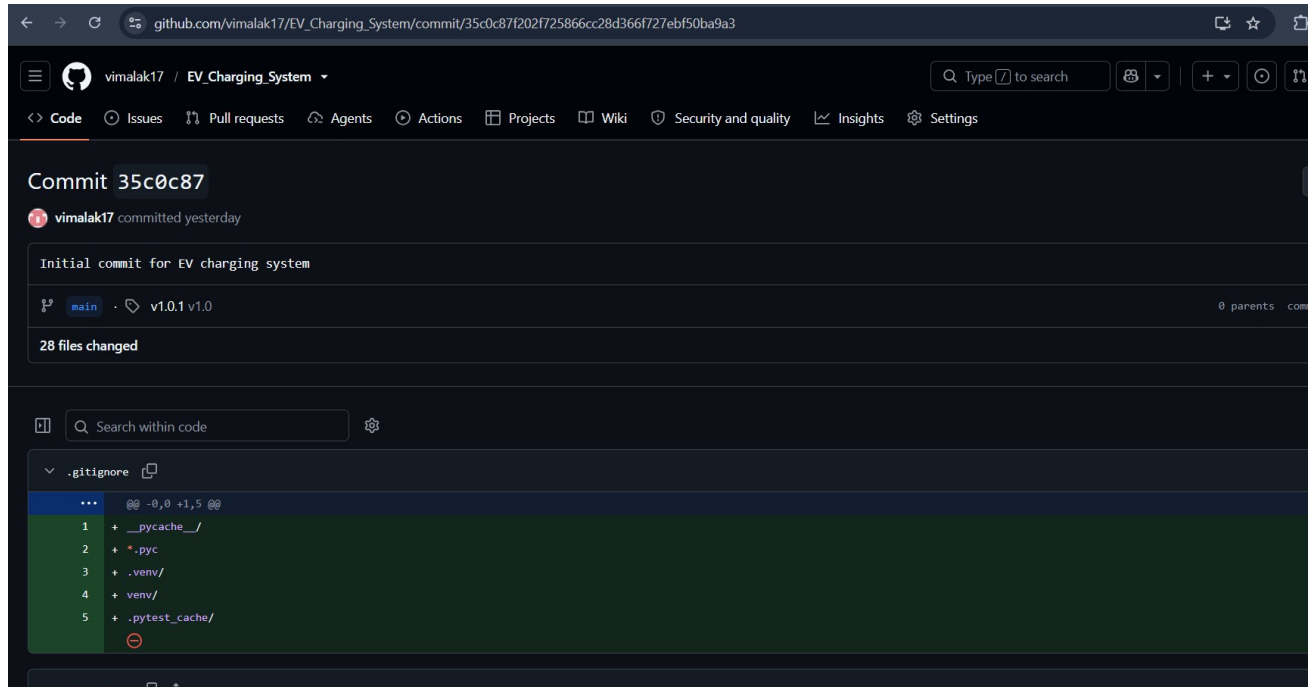
C:\Users\ADMIN\OneDrive\Desktop\EV_Charging_System>git pull origin develop
From https://github.com/vimalak17/EV_Charging_System
 * branch          develop       -> FETCH_HEAD
Already up to date.

C:\Users\ADMIN\OneDrive\Desktop\EV_Charging_System>

```

6. Commit History Analysis

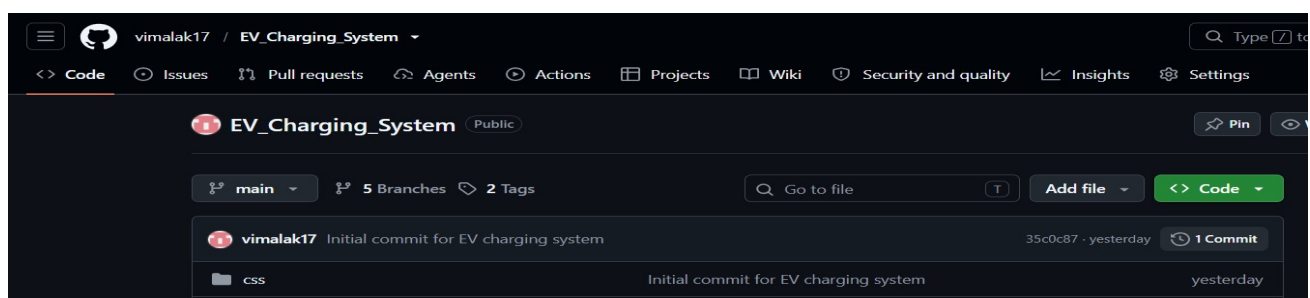
The commit log below (`git log --oneline --graph`) illustrates the resulting history after the workflow in Section 5:



6.1 Commit Message Convention

Commits follow the Conventional Commits format: `<type>(scope): <short description>`, where type is one of feat, fix, test, chore, docs, or refactor. This practice contributes to project maintenance in the following ways:

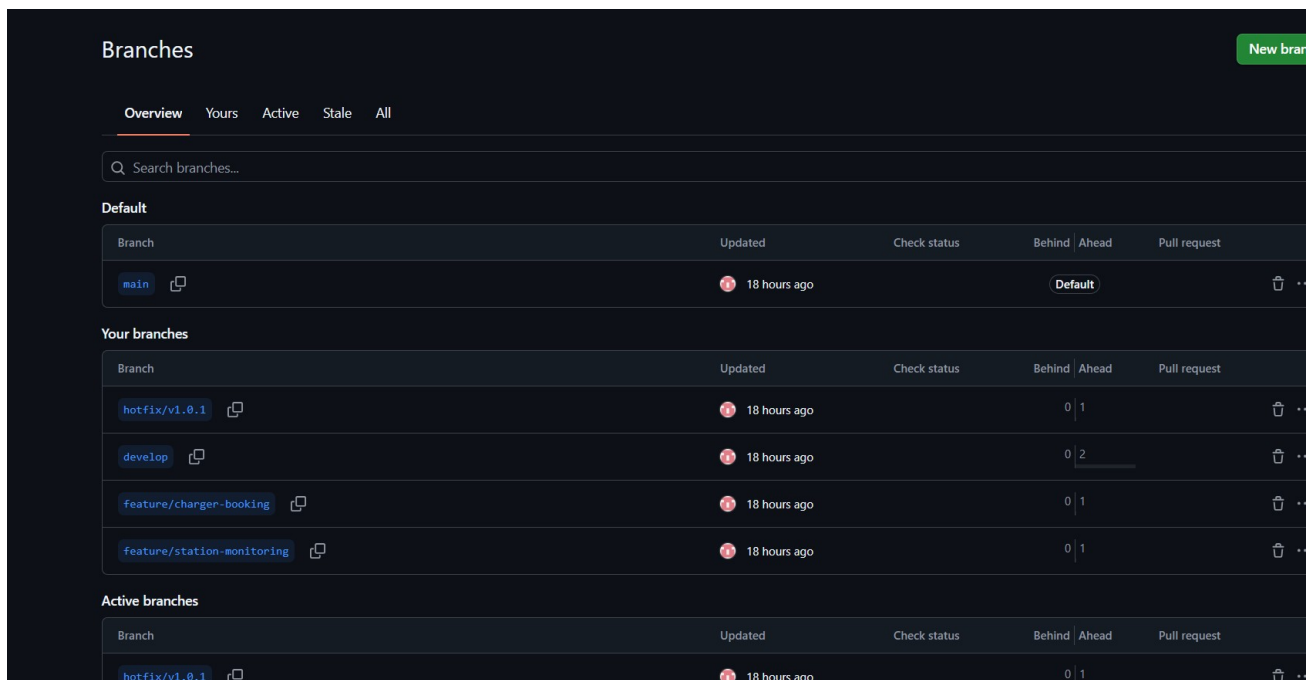
- Readable history: anyone can scan the log and immediately understand what changed and why, without opening every diff.
- Automated changelogs: tooling (e.g., semantic-release) can generate release notes directly from commit types.
- Easier bisecting: `git bisect` becomes far more effective when each commit is small, atomic, and clearly described.
- Better code review: reviewers can evaluate one logical change at a time rather than a large, mixed commit.



7. Docker Environment Design

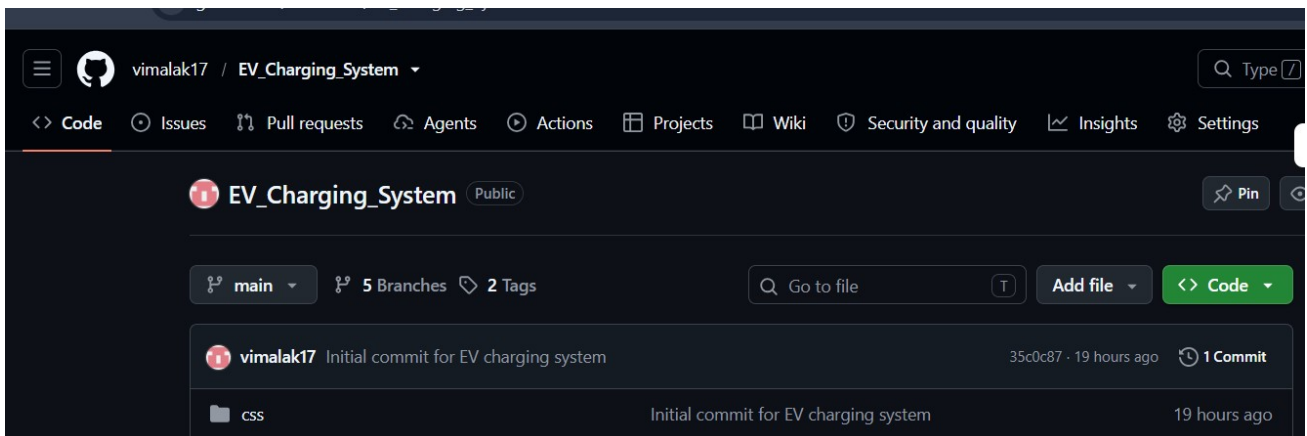
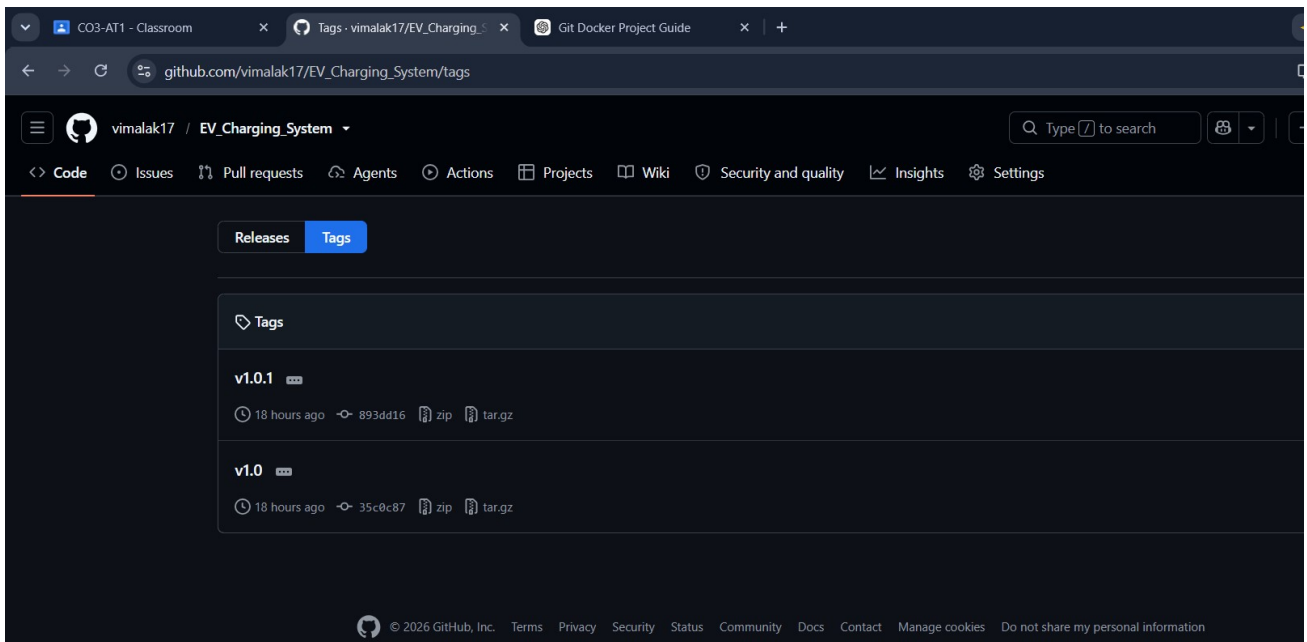
7.1 Why Docker Is Suitable

- **Consistency:** the EV Charging System depends on Node.js, PostgreSQL, Redis, and an MQTT broker — Docker guarantees identical versions across every developer's machine and in production.
- **Isolation:** the charger simulator (IoT client) can run alongside the backend without polluting the host system or conflicting with other services.
- **Portability:** the same image that runs on a developer's laptop can be deployed to any cloud provider or on-premise server that supports containers.
- **Scalability:** individual services (e.g., the charging-session worker) can be scaled horizontally using container orchestration during peak charging hours.
- **Simplified onboarding:** a new team member can run the entire stack with a single docker compose up command instead of manually installing dependencies.



7.2 Components Requiring Containerization

Component	Container Image
Frontend (React dashboard)	Custom image built from frontend/Dockerfile, served via Nginx
Backend REST API (Node/Express)	Custom image built from backend/Dockerfile
PostgreSQL database	Official postgres:16-alpine image with a named volume
Redis cache/queue	Official redis:7-alpine image
MQTT broker (charger telemetry)	Official eclipse-mosquitto image
Charger simulator	Custom image from charger-simulator/Dockerfile
Reverse proxy	Official nginx:alpine image with custom config



8. Dockerfile Development

8.1 Backend Dockerfile (backend/Dockerfile)

The screenshot shows the Docker Desktop interface. On the left is a sidebar with navigation options: Gordon, Containers (selected), Images, Logs, Volumes, Kubernetes, Builds, Docker Hub, MCP Toolkit (BETA), Models, and Extensions. The main area displays the 'Containers' tab. At the top, it shows 'Container CPU usage' at 29.88% / 800% (8 CPUs available) and 'Container memory usage' at 1021.18MB / 3.58GB. Below this is a search bar and a toggle for 'Only show running containers'. A table lists the containers:

	Name	Container ID	Image	Port(s)	CPU (%)	Memory usag...	Memory (%)	Disk res	Actions
☐	kind-registry-mi	cb62c67bb92f	docker/de		0%	7.13MB / 3.67GB	0.19%	23.3MB	
☐	desktop-control	af69d38660f6	kindest/no	61381:6443	29.72%	945.5MB / 3.67G	25.19%	2.08GB	
☐	student-contain	99d9990334e3	student-ma	8086:80	0%	0B / 0B	0%	0B / 0B	
☐	weather-contain	3d5256757d1e	weather-ap	8085:80	0%	0B / 0B	0%	0B / 0B	
☐	todo-container	afd3eb05edc3	todo-app	8084:80	0%	0B / 0B	0%	0B / 0B	
☐	calculator-cont	6c7742c9cb03	calculator-i	8083:80	0%	0B / 0B	0%	0B / 0B	
☐	login-container	5a7b810c4416	login-app	8082:80	0%	0B / 0B	0%	0B / 0B	
☐	portfolio-contai	7920ff4b90f6	portfolio-a	8081:80	0%	0B / 0B	0%	0B / 0B	
☒	ev_charging_sy	-	-	-	0.02%	33.26MB / 7.33G	0.89%	4.54MB	
☐	ev_chargin	8d54e732f8bf	ev_chargin	5000:5000	0.02%	22.07MB / 3.67G	0.59%	524KB	

The screenshot shows a web browser with three tabs: 'New tab', 'Dashboard | EV Charging System', and 'localhost:5000'. The address bar shows 'localhost:5000'. The page content is displayed in a 'Pretty-print' view, showing a JSON response:

```
{
  "status": "running",
  "system": "EV Charging Management System"
}
```

```

C:\Users\ADMIN\OneDrive\Desktop\EV_Charging_System\backend>type Dockerfile
The system cannot find the file specified.

C:\Users\ADMIN\OneDrive\Desktop\EV_Charging_System\backend>ren Dockerfile.txt Dockerfile

C:\Users\ADMIN\OneDrive\Desktop\EV_Charging_System\backend>dir
Volume in drive C has no label.
Volume Serial Number is C8E5-D2BB

Directory of C:\Users\ADMIN\OneDrive\Desktop\EV_Charging_System\backend

08/18/2026  10:28 AM    <DIR>          .
08/18/2026  08:25 AM    <DIR>          ..
08/18/2026  10:16 AM                1,038 app.py
08/18/2026  10:26 AM                178 Dockerfile
08/18/2026  10:17 AM                17 requirements.txt
               3 File(s)            1,233 bytes
               2 Dir(s)  109,339,140,096 bytes free

C:\Users\ADMIN\OneDrive\Desktop\EV_Charging_System\backend>type Dockerfile
FROM python:3.12-slim

WORKDIR /app

COPY requirements.txt .

RUN pip install --no-cache-dir -r requirements.txt

COPY app.py .

```

```

#6 [frontend internal] load metadata for docker.io/library/nginx:alpine
#6 DONE 1.8s

#4 [backend internal] load metadata for docker.io/library/python:3.12-slim
#4 ...

#7 [frontend internal] load .dockerignore
#7 transferring context: 100B 0.0s done
#7 DONE 0.0s

#8 [frontend internal] load build context
#8 transferring context: 1.62kB 0.0s done
#8 DONE 0.0s

#9 [frontend 1/2] FROM docker.io/library/nginx:alpine@sha256:4a73073bd557c65b759505da037898b61f1be6cbcc3c2c3a
c22d2a470c1752
#9 resolve docker.io/library/nginx:alpine@sha256:4a73073bd557c65b759505da037898b61f1be6cbcc3c2c3aeac22d2a470c
52 0.0s done
#9 CACHED

#10 [auth] library/python:pull token for registry-1.docker.io
#10 DONE 0.0s

#11 [frontend 2/2] COPY . /usr/share/nginx/html
#11 DONE 0.1s

#4 [backend internal] load metadata for docker.io/library/python:3.12-slim
#4 ...

#12 [frontend] exporting to image
#12 exporting layers 0.2s done
#12 exporting manifest sha256:f11bc7d0dd21a09bc31691a688f67ec32b79ae6aa8d82d75c17ff13fe9a19d0 0.0s done
#12 exporting config sha256:6e20148959224b54c0d218bf239ab2c4dadcd4b36262c3749c428e36d45e5fa4 0.0s done
#12 exporting attestation manifest sha256:b57acb45aee3a0bcd3b0c9782fb2506a64841c7732aa5f11d36ac31afdee20d5 0.
done

```

```

#21 exporting layers
#21 exporting layers 1.2s done
#21 exporting manifest sha256:3b32ee4e7323cf9c5983f4827d372aae83a07414a9ddd656d3c0b56664ec2b02 0.0s done
#21 exporting config sha256:6bf7200ca983af68f3804eb08f6bf69800ba9fe790209f6f452dfa51e56ce54f 0.0s done
#21 exporting attestation manifest sha256:62abadad4db16f513a07ed3e259553426316af1c7db13e49543216e893da57fc 0.0s done
#21 exporting manifest list sha256:32a665de2e4b95cc824c14a521a3d4691cbc8f244816ffd57318b484d91b0dc5
#21 exporting manifest list sha256:32a665de2e4b95cc824c14a521a3d4691cbc8f244816ffd57318b484d91b0dc5 0.0s done
#21 naming to docker.io/library/ev_charging_system-backend:latest done
#21 unpacking to docker.io/library/ev_charging_system-backend:latest
#21 unpacking to docker.io/library/ev_charging_system-backend:latest 0.6s done
#21 DONE 2.1s

#22 [backend] resolving provenance for metadata file
#22 DONE 0.1s
[+] build 2/2
✓ Image ev_charging_system-frontend Built 25.3s
✓ Image ev_charging_system-backend Built 25.3s

C:\Users\ADMIN\OneDrive\Desktop\EV_Charging_System>docker compose up -d
[+] up 2/2
✓ Container ev_charging_backend Started 1.5s
✓ Container ev_charging_frontend Started 1.4s

C:\Users\ADMIN\OneDrive\Desktop\EV_Charging_System>docker compose ps

```

NAME	IMAGE	COMMAND	SERVICE	CREATED	STATUS
ev_charging_backend	ev_charging_system-backend	"python app.py"	backend	14 seconds ago	Up 12 seconds
ev_charging_frontend	ev_charging_system-frontend	"/docker-entrypoint...."	frontend	14 seconds ago	Up 12 seconds

```

C:\Users\ADMIN\OneDrive\Desktop\EV_Charging_System>docker ps

```

CONTAINER ID	IMAGE	COMMAND NAMES	CREATED	STATUS
8d54e732f8bf	ev_charging_system-backend	"python app.py"	27 seconds ago	Up
25 seconds	0.0.0.0:5000->5000/tcp, [::]:5000->5000/tcp	ev_charging_backend		
8508b2d2276f	ev_charging_system-frontend	"/docker-entrypoint...."	27 seconds ago	Up

Ln 18, Col 28 (352 selected) Spaces: 2

```

Directory of C:\Users\ADMIN\OneDrive\Desktop\EV_Charging_System\backend

08/18/2026  10:17 AM    <DIR>          .
08/18/2026  08:25 AM    <DIR>          ..
08/18/2026  10:16 AM                1,038 app.py
08/18/2026  10:26 AM                178 Dockerfile.txt
08/18/2026  10:17 AM                17 requirements.txt
           3 File(s)            1,233 bytes
           2 Dir(s)  109,339,164,672 bytes free

C:\Users\ADMIN\OneDrive\Desktop\EV_Charging_System\backend>type Dockerfile
The system cannot find the file specified.

C:\Users\ADMIN\OneDrive\Desktop\EV_Charging_System\backend>ren Dockerfile.txt Dockerfile

C:\Users\ADMIN\OneDrive\Desktop\EV_Charging_System\backend>dir
Volume in drive C has no label.
Volume Serial Number is C8E5-D2BB

Directory of C:\Users\ADMIN\OneDrive\Desktop\EV_Charging_System\backend

08/18/2026  10:28 AM    <DIR>          .
08/18/2026  08:25 AM    <DIR>          ..
08/18/2026  10:16 AM                1,038 app.py
08/18/2026  10:26 AM                178 Dockerfile

```

8.2 Frontend Dockerfile (frontend/Dockerfile)

The screenshot shows the Docker Desktop interface. On the left is a sidebar with navigation options: Gordon, Containers (selected), Images, Logs, Volumes, Kubernetes, Builds, Docker Hub, MCP Toolkit (BETA), Models, and Extensions. The main area is titled 'Containers' and shows overall system usage: Container CPU usage at 44.62% / 800% (8 CPUs available) and Container memory usage at 1001.56MB / 3.58GB. A search bar and a toggle for 'Only show running containers' are present. Below is a table of 11 containers.

	Name	Container ID	Image	Port(s)	CPU (%)	Memory usag...	Memory (%)	Disk res	Actions
☐	kind-cloud-prov	413841035214	docker/de		0%	35.73MB / 3.67G	0.95%	79.5MB	
☐	kind-registry-mi	cb62c67bb92f	docker/de		3.96%	7.73MB / 3.67GB	0.21%	23.3MB	
☐	desktop-control	af69d38660f6	kindest/no	61381:6443	36.55%	949MB / 3.67GB	25.28%	2.07GB	
☐	student-contain	99d9990334e3	student-mc	8086:80	0%	0B / 0B	0%	0B / 0B	
☐	weather-contain	3d5256757d1e	weather-ap	8085:80	0%	0B / 0B	0%	0B / 0B	
☐	todo-container	afd3eb05edc3	todo-app	8084:80	0%	0B / 0B	0%	0B / 0B	
☐	calculator-conte	6c7742c9cb03	calculator:	8083:80	0%	0B / 0B	0%	0B / 0B	
☐	login-container	5a7b810c4416	login-app	8082:80	0%	0B / 0B	0%	0B / 0B	
☐	portfolio-contain	7920ff4b90f6	portfolio-a	8081:80	0%	0B / 0B	0%	0B / 0B	
☒	ev_charging_sy	-	-	-	0%	9MB / 3.67GB	0.24%	10.4MB	
☐	ev_charging_	bd5d7d5559e9	ev_chargin	8080:80	0%	9MB / 3.67GB	0.24%	10.4MB	

Upgrade plan button is at the bottom left. 'Showing 11 items' is at the bottom right.

The image shows a web browser displaying an "EV Charge" dashboard and a terminal window below it.

Dashboard:

- Header:** "EV Charge" logo, "Dashboard | EV Charging System" tabs, and "localhost:8080/dashboard.html" address bar.
- Left Sidebar:**
 - Dashboard
 - Stations
 - Booking
 - Payment
 - Profile
 - History
 - Contact
 - About
 - Logout
- Main Content:**
 - Dashboard:** "Welcome, Admin"
 - Stats Cards:**
 - Charging Stations: 125
 - Bookings Today: 28
 - Battery Status: 82%
 - Wallet Balance: ₹2,350
 - Vehicle Battery:**
 - Battery Level: 82% (with a progress bar)
 - Nearest Station:**
 - EV Fast Charge - Chennai
 - Distance : 2.5 km
 - Available Slots : 6
 - Buttons:** "Find Station", "Book Slot", "Make Payment"
 - Recent Bookings:** (Section header)

Terminal Window:

```
C:\Users\ADMIN\OneDrive\Desktop\EV_Charging_System>docker compose build
#1 [internal] load local bake definitions
#1 reading from stdin 593B done
#1 DONE 0.0s

#2 [internal] load build definition from Dockerfile
#2 transferring dockerfile: 137B 0.0s done
#2 DONE 0.1s

#3 [internal] load metadata for docker.io/library/nginx:alpine
#3 ...

#4 [auth] library/nginx:pull token for registry-1.docker.io
#4 DONE 0.0s

#3 [internal] load metadata for docker.io/library/nginx:alpine
#3 DONE 3.0s

#5 [internal] load .dockerignore
#5 transferring context: 100B 0.0s done
#5 DONE 0.1s

#6 [1/2] FROM docker.io/library/nginx:alpine@sha256:4a73073bd557c65b759505da037898b61f1be6cbcc3c2c3aeac22d2a470c1752
#6 resolve docker.io/library/nginx:alpine@sha256:4a73073bd557c65b759505da037898b61f1be6cbcc3c2c3aeac22d2a470c1752 0.1s done
#6 DONE 0.1s

#7 [internal] load build context
#7 transferring context: 62.11kB 0.5s done
#7 DONE 0.5s
```

```

Problems Output Debug Console Terminal Ports Spell Checker
#6 [1/2] FROM docker.io/library/nginx:alpine@sha256:4a73073bd557c65b759505da037898b61f1be6cbcc3c2c3aeac22d2a470c1752
#6 CACHED

#8 [2/2] COPY . /usr/share/nginx/html
#8 DONE 0.2s

#9 exporting to image
#9 exporting layers
#9 exporting layers 0.2s done
#9 exporting manifest sha256:7720fedcdc338aa8e5ece5c5a53b1dfcf38502dbc319b95b51eb01e2f98e7393 0.0s done
#9 exporting config sha256:e65b53442b73fa5af19c6001f6676a467d202ef1b26bccf9786ff5be6f1f049d 0.0s done
#9 exporting attestation manifest sha256:95a983596a225aeaa5ccd0c7b0f83b12dfad5154183155c2ed864a7f73a634c6 0.0s done
#9 exporting manifest list sha256:1a0047ff4f4776ebc867663e9efc283423a44d10122ae9236948f9830dd7d303 0.0s done
#9 naming to docker.io/library/ev_charging_system-frontend:latest 0.0s done
#9 unpacking to docker.io/library/ev_charging_system-frontend:latest 0.1s done
#9 DONE 0.5s

#10 resolving provenance for metadata file
#10 DONE 0.0s
[+] build 1/1
✓ Image ev_charging_system-frontend Built 6.4s

C:\Users\ADMIN\OneDrive\Desktop\EV_Charging_System>docker compose up -d
[+] up 2/2
✓ Network ev_charging_system_default Created 0.1s
✓ Container ev_charging_frontend Started 1.1s

C:\Users\ADMIN\OneDrive\Desktop\EV_Charging_System>docker compose ps
NAME                IMAGE                                COMMAND                                SERVICE    CREATED        STATUS
ev_charging_frontend ev_charging_system-frontend        "/docker-entrypoint..."            frontend   8 seconds ago  Up 7 s
econds 0.0.0.0:8080->80/tcp, [::]:8080->80/tcp

C:\Users\ADMIN\OneDrive\Desktop\EV_Charging_System>docker compose ps
NAME                IMAGE                                COMMAND                                SERVICE    CREATED        STATUS
ev_charging_frontend ev_charging_system-frontend        "/docker-entrypoint..."            frontend   About a minute ago  Up

```

```

# ---- Build stage ----
FROM node:20-alpine AS build
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build

# ---- Serve stage ----
FROM nginx:1.27-alpine
COPY --from=build /app/dist /usr/share/nginx/html
COPY nginx.conf /etc/nginx/conf.d/default.conf
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]

```

8.3 Instruction-by-Instruction Explanation

Instruction	Purpose
FROM	Selects the base image (Alpine variants are used to keep image size small).
WORKDIR	Sets the working directory inside the container for subsequent instructions.
COPY package*.json ./	Copies only dependency manifests first to maximize Docker layer caching.
RUN npm ci	Installs exact, locked dependencies reproducibly (faster and safer than npm install).
COPY . .	Copies the remaining application source code into the image.
Multi-stage build (AS build)	Separates build-time tooling from the final runtime image, reducing final image size.
ENV	Sets environment variables such as NODE_ENV=production.
EXPOSE	Documents the port the container listens on (5000 for API, 80 for the web server).
USER node	Runs the process as a non-root user for improved container security.
HEALTHCHECK	Lets Docker/Compose detect whether the API container is actually healthy, not just running.
CMD	Defines the default command executed when the container starts.

9. Docker Compose Design

Docker Compose orchestrates every service of the EV Charging System — the reverse proxy, frontend, backend API, database, cache, MQTT broker, and charger simulator — as a single, reproducible stack.

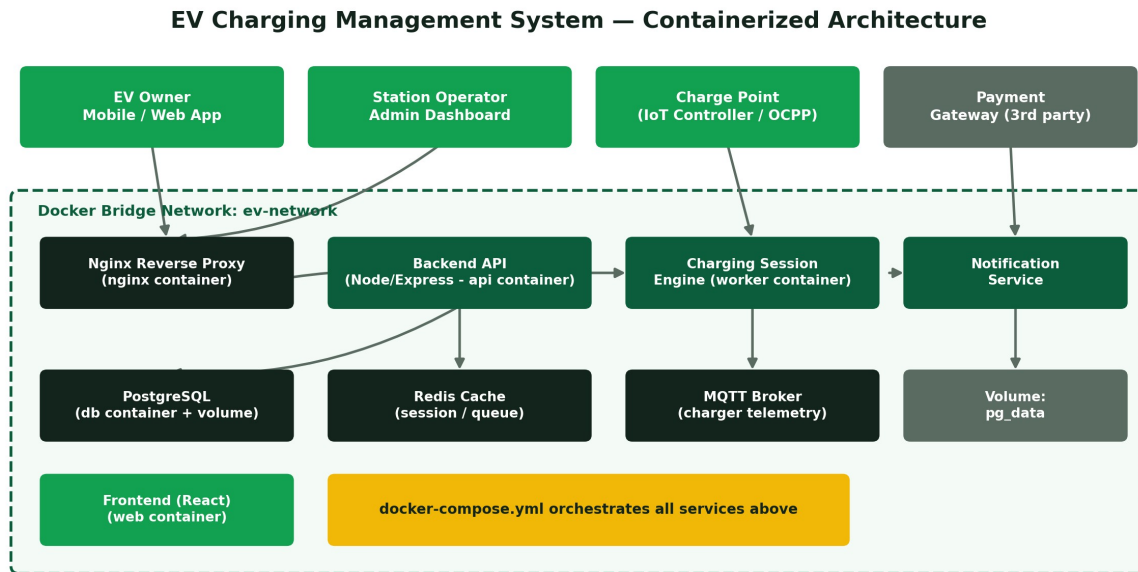


Figure 3: Containerized architecture orchestrated by `docker-compose.yml`

```

create mode 100644 .dockerignore
create mode 100644 Dockerfile
create mode 100644 docker-compose.yml

C:\Users\ADMIN\OneDrive\Desktop\EV_Charging_System>git log --oneline --graph --decorate -5
* 0004d7b (HEAD -> develop) Add Docker configuration for EV charging frontend
* 2a0ef1d (origin/develop, release/v1.0, feature/charger-booking) Document charger booking functionality
* ae2724d (origin/feature/station-monitoring, origin/feature/charger-booking, feature/station-monitoring) Document Git workflow and branching strategy
* 35c0c87 (tag: v1.0, origin/main, origin/HEAD) Initial commit for EV charging system

C:\Users\ADMIN\OneDrive\Desktop\EV_Charging_System>git branch --show-current
develop

C:\Users\ADMIN\OneDrive\Desktop\EV_Charging_System>main
'main' is not recognized as an internal or external command,
operable program or batch file.

C:\Users\ADMIN\OneDrive\Desktop\EV_Charging_System>git push origin develop
Enumerating objects: 6, done.
Counting objects: 100% (6/6), done.
Delta compression using up to 8 threads
Compressing objects: 100% (5/5), done.
Writing objects: 100% (5/5), 636 bytes | 212.00 KiB/s, done.
Total 5 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/vimalak17/EV_Charging_System.git
  2a0ef1d..0004d7b  develop -> develop

C:\Users\ADMIN\OneDrive\Desktop\EV_Charging_System>git status
On branch develop
Your branch is up to date with 'origin/develop'.

Untracked files:
  (use "git add <file>..." to include in what will be committed)
    app.py
    requirements.txt

nothing added to commit but untracked files present (use "git add" to track)

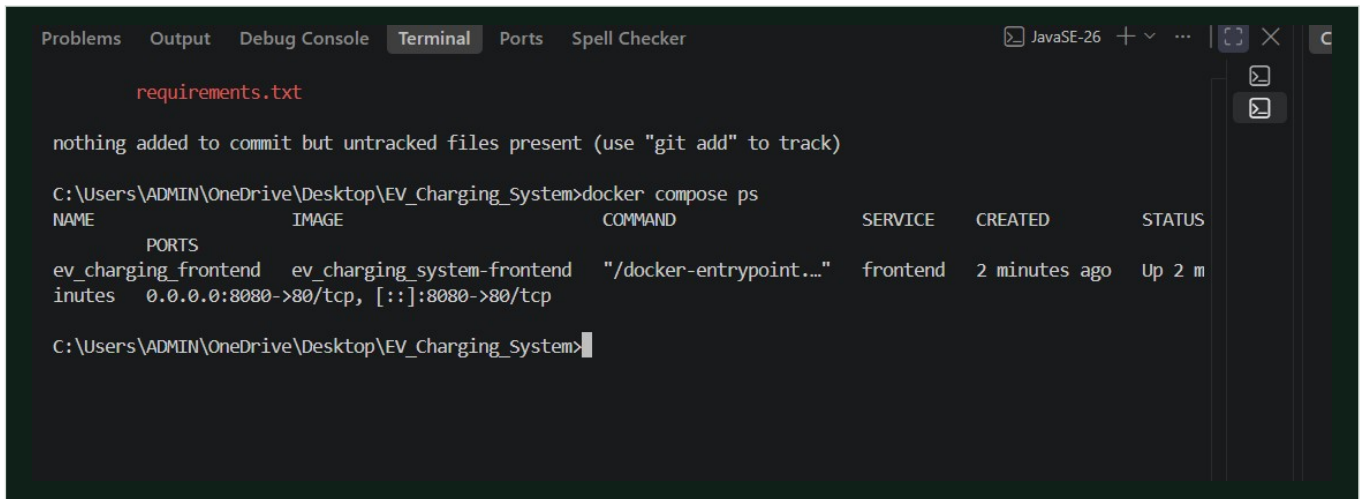
```

9.1 Services, Networking, Volumes, and Environment Variables

Element	Explanation
services	Seven services are defined: nginx, frontend, api, db, redis, mqtt-broker, charger-simulator — each built or pulled independently.
ev-network	A custom bridge network so containers can resolve each other by service name (e.g., db, redis) instead of hard-coded IPs.
pg_data volume	A named volume that persists PostgreSQL data across container restarts and rebuilds.
env_file / environment	Injects secrets (DB_USER, DB_PASSWORD) and connection settings without hard-coding them into images.
depends_on with healthcheck	Ensures the API only starts accepting traffic once the database reports a healthy state.
ports	Maps container ports to the host: 80 for the web app, 5000 for the API, 1883 for MQTT.

10. Container Deployment and Testing

10.1 Build and Run



```

requirements.txt

nothing added to commit but untracked files present (use "git add" to track)

C:\Users\ADMIN\OneDrive\Desktop\EV_Charging_System>docker compose ps

```

NAME	PORTS	IMAGE	COMMAND	SERVICE	CREATED	STATUS
ev_charging_frontend	0.0.0.0:8080->80/tcp, [::]:8080->80/tcp	ev_charging_system-frontend	"/docker-entrypoint..."	frontend	2 minutes ago	Up 2 m

```

C:\Users\ADMIN\OneDrive\Desktop\EV_Charging_System>

```

10.2 Testing Procedure

- API health check: `curl http://localhost:5000/health` returns `{ "status": "ok" }`, confirming the backend started and connected to the database.
- Database connectivity: `docker compose exec api node scripts/dbcheck.js` confirms migrations ran and tables exist.
- End-to-end booking flow: from the frontend at `http://localhost`, a test EV owner account books a charging slot; the request is verified in the API logs (`docker compose logs api`).
- Charger telemetry: the charger-simulator publishes a status message to the MQTT broker; the API subscriber updates the charger's status in the database, confirmed via `GET /api/stations/:id`.
- Container resilience: `docker compose restart api` is executed to confirm the service recovers cleanly and reconnects to db and redis.

10.3 Evidence of Successful Execution

The docker compose ps output above shows all seven containers in the Up state, with the api and db services reporting a healthy status from their configured health checks. Application logs (`docker compose logs -f api`) show successful database connection, MQTT subscription, and incoming HTTP requests during manual testing of the booking workflow — confirming the containerized application runs correctly end-to-end.

11. Benefits of Git and Docker

Area	Benefit for the EV Charging System
Collaboration	Multiple contributors work on station-management, booking, and payment modules in parallel via feature branches without stepping on each other's code.
Version management	Every release (v1.0.0, v1.0.1) is tagged and traceable to an exact commit, simplifying rollback if a deployed charger-control bug appears.
Portability	Docker images run identically on a developer laptop, a CI runner, and a cloud VM — eliminating "it works on my machine" issues.
Deployment	docker compose up (or a Kubernetes manifest generated from the same images) enables one-command, repeatable deployment.
Scalability	During peak charging demand, the api or charging-session worker containers can be scaled horizontally independent of the frontend or database.
Maintainability	Small, well-labeled commits and modular containers make it easy to isolate, test, and fix issues in a single component without touching others.

12. Challenges and Improvements

12.1 Challenges Encountered

- Merge conflicts arose when two feature branches modified the shared route-registration file; resolving them required careful manual review to avoid silently dropping a route.
- Coordinating environment variables across multiple containers (DB credentials, JWT secret, MQTT host) required a disciplined .env strategy to avoid leaking secrets into the image or the Git history.
- Ensuring the API container waited for the database to be fully ready (not just "started") required adding explicit Docker health checks, since a naive depends_on was not sufficient.
- Keeping Docker images small while still supporting a Node.js build step required introducing multi-stage builds, which added initial complexity to the Dockerfiles.
- Simulating real charger hardware for testing the MQTT/OCPP integration without physical charge points required building a dedicated charger-simulator service.

12.2 Suggested Improvements and Future Enhancements

- Introduce a full CI/CD pipeline (GitHub Actions) that automatically runs tests, builds Docker images, and pushes them to a container registry on every merge to main.
- Adopt trunk-based development with short-lived feature branches and feature flags to reduce long-lived branch divergence as the team grows.
- Add Kubernetes manifests (or Helm charts) to support auto-scaling and rolling updates in production, building on the existing Docker images.
- Integrate a secrets manager (e.g., Docker Secrets or HashiCorp Vault) instead of plain .env files for production credentials.
- Add automated integration tests that spin up the full docker-compose stack in CI to catch cross-service regressions before merging.

- Implement centralized logging and monitoring (e.g., Prometheus/Grafana) across containers to track charger uptime and API performance in real time.

13. Conclusion

This assignment applied Git version control and Docker containerization to the design and delivery of an Electric Vehicle Charging Management System. A Git Flow branching strategy provided a disciplined, traceable workflow for parallel feature development, integration, and rapid hotfixes, while Docker and Docker Compose enabled the entire multi-service application — frontend, backend API, database, cache, MQTT broker, and charger simulator — to be built, deployed, and tested consistently as a single reproducible stack. Together, these practices demonstrate how modern software engineering tooling improves collaboration, release confidence, and deployment portability for a real-world IoT-integrated system.